Week 8: Building the Frontend: React 4

Problem Statement

This week you will evolve the **ThreadHive** frontend from React Context–based state management to **Redux Toolkit** — the industry-standard approach for managing complex global state in React applications. You will implement a Redux slice with async thunks, wire components to the Redux store, and use GitHub Copilot to generate additional slices, set up integration testing, generate project documentation, and refactor a codebase.

In this assignment, you will implement the `authSlice` from scratch, connect the `Login` and `Header` components to the Redux store, and then use AI tools — including **Agent Mode**, a **Background Agent**, and a **Cloud Agent** — to generate Redux slices, plan and execute integration testing, and refactor a codebase from Context API to Redux.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. The project includes code for the backend and the frontend. You do not need to modify the backend for this assignment, but you will need to have it running locally to test the frontend features you build. Open the backend and the frontend folders in separate VS Code windows. Since we will be making changes only to the frontend code, we can isolate it in its own VS Code window to ensure that the agent is not fed unnecessary context.
3. To get the backend running, in the VS Code window where you have the backend folder open, follow these steps:
    - Open a terminal in the backend folder.
    - Install dependencies: `npm install`
    - Populate the database: `npm run populate`
    - Start the server: `npm run dev`

    Ensure that you have set up the .env file appropriately for the backend.
4. Open the VS Code window where you have the frontend folder open.
    - Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
    - As you make changes throughout this tutorial, remember to regularly commit your work.

The main parts of the frontend codebase are as follows:

```
src/
├── App.jsx                    Root component with routing and Redux
Provider
├── main.jsx                   Entry point that renders the App component
├── api/                       Axios instance with interceptors for auth
tokens
├── config/                    API endpoint configurations and constants
├── components/                Reusable UI components (Header, Footer,
ThreadCard, CommentList, Forms, etc.)
├── pages/                     Page components (Auth, User)
├── reducers/                  Redux slices for auth, threads, comments,
subreddits, and theme
├── services/                  API service modules (authService,
threadService, commentService, etc.)
├── store/
│   └── store.js               Redux store configuration
└── utils/
    └── handleApiError.js      Error handling utility
```

This session is structured in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Redux slice (`authSlice`), wiring `Login.jsx` and `Header.jsx` to Redux store	No AI Tools
Part 2	AI-Assisted Development	Slice generation, integration testing, README generation, Cloud Agent codebase refactoring	GitHub Copilot

In **Part 1** you will implement the authentication Redux slice manually, without AI assistance. This ensures you understand how Redux Toolkit slices, async thunks, and the `extraReducers` builder pattern work together.

In **Part 2** you will use GitHub Copilot across three different AI surfaces — **Agent Mode** in your IDE, a **Background Agent** for asynchronous task delegation, and a **Cloud Agent** on GitHub for large-scale codebase refactoring.

By the end of this session, you will be able to:

- Create Redux Toolkit slices with async thunks for API calls
- Connect React components to the Redux store using `useSelector` and `useDispatch`
- Use Agent Mode to generate Redux slices from existing patterns
- Delegate well-scoped tasks (Integration Testing and Accessibility Audit in our case) to a Background Agent
- Use a Cloud Agent to perform multi-file refactoring on a codebase on Github

Part 1: Manual Implementation

In this part, you will implement the `authSlice.js` Redux slice and connect the `Login` and `Header` components to the Redux store. No AI tools should be used in this section.

1.1 Project Setup

Open a terminal in the root of the assignment folder and run:

```
npm install
```

Start the development server to confirm everything is working:

```
npm run dev
```

Open `http://localhost:5173` in your browser. You will see an empty page (with errors in the DevTools console) because `authSlice.js` is empty — that is expected until you implement it.

Before you start coding, take a moment to review the existing codebase:

1. Open `src/store/store.js` — this is the Redux store configuration. It imports reducers from all the slice files, but `authSlice` and `commentSlice` are empty stubs.
2. Open `src/reducers/currentThreadSlice.js` — this is a **complete slice** that you can use as a reference for the pattern you will follow. Study the structure: imports, initial state, async thunks, slice definition with `reducers` and `extraReducers`.
3. Open `src/services/authService.js` — these service functions are already implemented and handle the API calls using the Axios library. Your slice will call these functions inside async thunks.

1.2 Implement `authSlice.js`

Implement the authentication slice in `src/reducers/authSlice.js`:

- Define initial state
- Create `loginUser` and `registerUser` async thunks
- Create (non-async) thunks for logout and saving user data locally
- Create the auth slice with `createSlice`
- Implement reducers for logout, clearAuthState, setUser
- Handle pending, fulfilled, and rejected states for the async thunks `loginUser` and `registerUser` in `extraReducers`

1.3 Wire Up `Login.jsx`

In `src/pages/Auth/Login.jsx`:

- Initialize `dispatch` with `useDispatch()`
- Dispatch the `loginUser` thunk with the form data on submit

1.4 Wire Up `Header.jsx`

In `src/components/Header/Header.jsx`:

- Initialize `dispatch` with `useDispatch()`
- In `handleLogout`, dispatch the logout action from `authSlice`

After completing this task, verify the full authentication flow:

1. Open the app. You should now be redirected to the login page.
2. Click the `Register` button and create a new account.
3. Login with the newly created account — you should be redirected to the home page
4. Refresh the page while logged in — you should remain logged in (localStorage persistence)
5. Click **Logout** in the header — you should be redirected to the login page

We have now successfully used Redux Toolkit for authentication state management in the ThreadHive frontend. This completes Part 1.

Part 2: AI-Assisted Development with GitHub Copilot

In this part, you will use GitHub Copilot across **three different AI surfaces** to perform a range of tasks. Let us begin by setting up custom instructions for our codebase.

2.1 Setting up Custom Instructions

Generate custom instructions (a `AGENTS.md` file) using the `/init` command.

Carefully review the generated instructions and edit them if necessary. Ensure that they include appropriate instructions on the use of Redux Toolkit for state management.

If using previous version of the instructions, replace the **State Management** section with instructions like the following:

```
#### State Management (Redux Toolkit)

1. All global state is managed with Redux Toolkit. The store is configured
in `src/store/store.js`.
2. Each feature has its own slice in `src/reducers/` (e.g., `authSlice.js`,
`threadListSlice.js`).
3. Use `useSelector` to read state and `useDispatch` to dispatch actions.
```

```
Never import the store directly in components.
4. For API calls, use `createAsyncThunk`. Each thunk calls a function from
`src/services/`.
5. Handle loading, success, and error states in `extraReducers` using the
builder pattern:
   - `.addCase(thunk.pending, ...)` → set loading true, clear error
   - `.addCase(thunk.fulfilled, ...)` → update state with payload, set
loading false
   - `.addCase(thunk.rejected, ...)` → set loading false, store error from
payload
6. Never access `localStorage` directly in components — always use Redux
actions (`loginUser`, `logout`, `setUser`).
7. Use `handleApiError` from `src/utils/handleApiError.js` in thunk catch
blocks for consistent error handling.
```

Save the file. These updated instructions ensure that Copilot-generated code uses latest Redux Toolkit patterns.

2.2 Generate Redux Slices with Agent Mode

The `commentSlice.js` file is currently an empty stub. Instead of writing them manually, you will use Copilot's **Agent Mode** to generate it based on the `authSlice` you implemented in Part 1.

Generate `commentSlice.js`

First, open `src/store/store.js` and **uncomment the `commentReducer` import and the `comments` reducer** in the store configuration.

Open a new Copilot Chat and switch to **Agent Mode**. Use the following prompt:

```
Implement the commentSlice in #file:src/reducers/commentSlice.js following
the same pattern.

Implement the following components in the slice:
1. Initial state:
  - comments (array),
  - loading (boolean),
  - error (null)
2. Async thunks:
  - fetchComments
  - addComment
  - upvoteCommentThunk
  - downvoteCommentThunk
3. Synchronous reducer:
  - clearComments — resets state to initialState
4. extraReducers: add pending/fulfilled/rejected cases for all async thunks
```

Review carefully and accept. The model must have identified and picked up the relevant service functions from `src/services/commentService.js`.

Import and use the appropriate actions from this slice in the following components:

- `ThreadPage.jsx`: to fetch, add and clear comments.
- `CommentList.jsx`: to display, upvote, and downvote comments.

After modifying the above files, verify the app works end-to-end:

1. Log in and confirm the home feed loads threads
2. Navigate to a thread page and confirm comments load
3. Post a new comment and confirm it appears in the list
4. Upvote and downvote a comment and confirm the UI updates accordingly

2.3 Integration Testing and Accessibility Audit with Background Agents

In this task, you will use **Plan Mode** to design the integration testing strategy, hand off the implementation to a **Background Agent**, and then launch a **second Background Agent** to perform an accessibility audit — both running asynchronously while you continue with other tasks.

What is a Background Agent?

A Background Agent is a Copilot-powered agent that runs tasks **asynchronously in a separate session** within VS Code. You assign it a task, and it works independently — creating a branch, making changes, and notifying you when it's done. You can continue coding in your main session while the agent works.

Background agents in VSCode are powered by Copilot CLI. Because they rely on the CLI these sessions continue running even if the VS Code window is closed. If you do not have Copilot CLI installed, use the following command to install it globally:

```
npm install -g @github/copilot-cli
```

You can verify the installation with:

```
copilot --version
```

This should print the installed version of Copilot CLI.

Step 1: Plan the Testing Strategy (Plan Mode)

Open a new Copilot Chat and switch to **Plan Mode**. Use the following prompt:

```
Plan a complete integration testing setup for this React + Redux Toolkit
frontend project.

I need a plan that covers:
1. Which testing libraries and tools to install (test runner, mocking,
assertions)
2. Configuration changes needed (vite.config.js, setup files)
3. Mock data structure (what entities to mock, what fields they need)
4. API mock handlers (which endpoints to intercept, success and error
cases)
5. Integration test suites to create:
   - Thread flow: fetch threads, create thread, upvote/downvote, fetch by
ID, handle not found
   - Comment flow: fetch comments for a thread, add comment,
upvote/downvote comment

Do not generate code — only produce a detailed plan with file paths and
specifications.
```

Review the plan Copilot generates. It should identify:

- **Dependencies**: Vitest, jsdom, MSW, @testing-library packages
- **Configuration**: Test block in `vite.config.js`, a setup file, MSW server setup
- **Mock data**: Users, threads, comments, subreddits, auth responses
- **Handlers**: All REST endpoints matching your service functions
- **Test files**: Separate suites for thread and comment flows

Once you are satisfied with the plan, you will hand it off to a Background Agent for execution.

Step 2: Hand Off to the Background Agent

To delegate a task to a Background Agent, open the **Copilot Chat** panel, change the session target from 'Local' to 'Copilot CLI', and then select 'Start Implementation' to begin implementing the above plan. VS Code will create a new session for this task, carrying over the conversation history and context.

You can see the new session in the session list. While the integration testing agent is working, you can continue with other tasks in your main session. The agent might ask for permissions to execute certain actions. You can approve all commands for this session to allow it to work uninterrupted.

Step 3: Launch a Second Background Agent — Accessibility Audit

Now that the integration testing agent is running, you will fire off a **second Background Agent** to perform an accessibility audit and fix issues across the component library. This task is fully independent — it modifies component JSX and CSS files, while the testing agent only creates new files in `tests/`. Both agents can work in parallel without conflicts.

Open a new **Background Agent** session and use the following prompt:

```
Perform an accessibility audit and apply fixes across all React components
in this project.
Audit and fix typical accessibility issues, including semantic HTML, ARIA
attributes, keyboard navigation, image alt text, and color contrast.

Constraints:
- Do NOT change component logic, state management, or API calls
- Do NOT restructure the file/folder layout
- Keep all existing class names and styling intact — only add or adjust,
never remove
```

Step 4: Generate a README (while the Background Agents work)

While both Background Agents are working in the background, you can continue working on other tasks. As an example, use **Agent Mode** to generate a `README.md` file for the project.

Open a new Copilot Chat in a Local agent, switch to **Agent Mode**, and use the following prompt:

```
Generate a professional README.md file for this project.
```

Review the generated README. Make sure it accurately reflects the project structure and doesn't include information about features that don't exist. Save it as `README.md` in the project root (replacing the existing placeholder if present).

Review the Background Agent's Work

The Background Agents will notify you in their respective sessions when they have completed their tasks. You can switch to those sessions, review the changes they made, and merge them into your main branch. Carefully review the tests generated by the integration testing agent and the accessibility fixes made by the second agent. Run the tests and manually verify the app's functionality to ensure there are no regressions.

2.4 Refactor a Codebase with a Cloud Agent

In this task, you will use GitHub Copilot's **Cloud Agent (Coding Agent)** to perform a multi-file refactoring on a separate codebase. The Cloud Agent runs entirely on GitHub — it spins up a cloud environment, reads the codebase, makes changes, and opens a Pull Request for your review.

What is a Cloud Agent?

A Cloud Agent (also called Copilot Coding Agent) is assigned tasks through **GitHub Issues**. When you assign an issue to Copilot, it:

1. Spins up a cloud-based development environment
2. Analyzes the codebase to understand the architecture
3. Makes code changes across multiple files
4. Opens a Pull Request with a description of all changes
5. You review, request changes, or merge — just like a human teammate's PR

This is ideal for well-scoped tasks that touch many files, like refactoring, migrations, or adding cross-cutting features.

The Task: Refactor the Week 7 ThreadHive Codebase

The Week 7 version of ThreadHive used the **Context API** (`AuthContext` + `useAuth()`) for state management and `fetch()` for API calls. You will ask the Cloud Agent to refactor it to use **Redux Toolkit** and **Axios** — the same patterns you implemented manually in Part 1 of this session.

Step 1: Fork and Clone the Week 7 Codebase

Fork the Week 7 ThreadHive codebase — **ThreadHive Week 7 Repository** — to your own GitHub account using the **Fork** button on GitHub. This creates a copy of the repository under your account where you have permissions to create branches and PRs.

Step 2: Create a GitHub Issue

Go to your forked repository on GitHub and create a new Issue with the following title and body:

Title:

```
Refactor: Migrate from Context API to Redux Toolkit and from fetch to Axios
```

Body:

```
## Overview

Refactor the ThreadHive frontend as follows:
1. Replace all Context API state management with Redux Toolkit slices and
store configuration
2. Replace all `fetch()` calls in the codebase with Axios, using a
centralized Axios instance with interceptors for auth tokens

## Constraints

- Do NOT change the UI or component structure — only the state management
```

```
and HTTP layers
- Maintain the same API response handling (the backend returns data in a
`data` property)
- Ensure localStorage persistence still works for auth state
- All existing routes and navigation should work unchanged
```

Step 3: Assign the Issue to Copilot

On the Issue page, assign it to **Copilot** by clicking the **Assignees** section and selecting `copilot` (or typing `@copilot` in a comment).

The Cloud Agent will begin working by creating a separate branch in your repository. You can track its progress in the Issue comments or in the Agents tab.

Step 4: Review the Pull Request

When the Cloud Agent finishes, it will open a **Pull Request** linked to the issue. Review it carefully using this checklist:

If you find issues, leave review comments on the PR and request changes. The Cloud Agent can iterate on feedback. Once everything looks good, **merge the PR**.